



Imersão GenAI para Desenvolvedores

Módulo 3

Preâmbulo: Memória e RAG

Versão: 1.1

Sumário

I	Exercício 1: Short Term Memory	2
II	Exercício 2: Memória Persistente	4
III	Exercício 3: Busca Semântica e Embeddings	6
IV	Exercício 4: Retrieval-Augmented Generation	8
V	Entrega e Avaliação entre Pares	10

Capítulo I

	Exercício : 01
Short Term Memory	
Pasta de entrega : <i>ex01/</i>	
Arquivos para entregar : <code>chatbot.py</code>	
Funções ou bibliotecas autorizadas : Biblioteca padrão, <code>requests</code> , <code>httpx</code> , SDKs oficiais de modelos	

Modelos de linguagem são **stateless**, ou seja, eles não mantêm memória entre chamadas. Cada requisição ao modelo é independente, e ele não “lembra” de interações anteriores. Para que possamos criar conversas coerentes, podemos atribuir contexto aos modelos fornecendo manualmente as mensagens anteriores, indicando quem falou em cada passo da conversa. Os papéis mais comuns são:

- **system**: define regras ou o comportamento esperado do modelo.
- **user**: representa a entrada ou pergunta do usuário.
- **assistant**: representa a resposta gerada pelo modelo.

Dessa forma, conseguimos simular uma memória, realimentando o modelo com as últimas interações.

1. Crie um Chatbot, que armazene e envie no prompt o histórico das cinco últimas interações com a IA.
2. Seu chatbot deve funcionar como um loop, realimentando a IA com as últimas interações ocorridas.
3. Seu chatbot deve encerrar com o comando 'bye'.

```
?> python3 chatbot.py
Q: Qual a capital da França?
A: Paris é a capital da França.

Q: E qual sua população?
A: A população de Paris é de aproximadamente 2,2 milhões de habitantes.
```

```
Q: bye  
?>
```



Use uma estrutura de dados para armazenar o histórico da conversa. Tanto Sua pergunta quanto a resposta da IA devem ser realimentadas em seu prompt.



Teste o chatbot fazendo perguntas encadeadas que dependem das respostas anteriores. Observe como a perda de memória afeta a coerência da conversa.

Capítulo II

	Exercício : 02
Memória Persistente	
Pasta de entrega : <i>ex02/</i>	
Arquivos para entregar : <code>persistent_chatbot.py</code>	
Funções ou bibliotecas autorizadas : Todas	

1. Implemente, no chatbot criado no exercício anterior, um sistema de persistência que mantém o histórico de mensagens compartilhadas entre usuário e assistente.
2. Além disso, implemente um sistema de resumos automáticos a cada 10 interações. Seu prompt deve combinar uma memória de curto prazo (últimas 5 mensagens) e memória de longo prazo (últimos 10 resumos) para fornecer contexto relevante a cada prompt.
3. Seu banco de dados deve manter a persistência entre as sessões de chat (se o usuário fechar o programa e retornar, seu histórico ainda deve fazer parte do contexto).
4. Abaixo, uma dica da estrutura de seu banco de dados, utilizando o [sqlalchemy](#). Não é necessário segui-la:

```
class Base(DeclarativeBase):
    pass

class Message(Base):
    __tablename__ = "messages"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    role: Mapped[str] = mapped_column(String(10), nullable=False) # 'user' or 'assistant'
    content: Mapped[str] = mapped_column(Text, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

class Summary(Base):
    __tablename__ = "summaries"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    content: Mapped[str] = mapped_column(Text, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
```



Manter resumos é uma estratégia eficaz para preservar o contexto sem sobrecarregar o prompt. Além de manter um prompt objetivo, diminui a quantidade de tokens enviados, contribuindo significativamente nos custos. Utilize a API do modelo para gerar resumos concisos das interações anteriores.

Capítulo III

	Exercício : 03
Busca Semântica e Embeddings	
Pasta de entrega : <i>ex03/</i>	
Arquivos para entregar : embeddings.py	
Funções ou bibliotecas autorizadas : Todas	

O conceito de **embeddings** é fundamental em processamento de linguagem natural (PLN) e machine learning. Em termos simples, embeddings são representações vetoriais (listas de números) de palavras, frases, documentos ou até mesmo entidades do mundo real. Esses vetores são projetados de modo que itens semanticamente semelhantes tenham representações vetoriais próximas no espaço vetorial. Isto permite uma [busca semântica](#), encontrando documentos com significado similar, mesmo que não compartilhem palavras exatas.

1. Utilizando a biblioteca [sentence_transformers](#), crie um programa que, dentre as frases abaixo, liste as três mais próximas semanticamente da palavra ou frase passada como argumento de linha de comando:

```
frases = [
    "O cachorro correu pelo parque atrás da bola azul.",
    "Ontem a bolsa de valores fechou em queda após anúncio do governo.",
    "O vulcão entrou em erupção, iluminando o céu noturno com lava.",
    "Aprender programação em Python pode abrir muitas portas no mercado de trabalho.",
    "O café recém-moído tem um aroma que desperta memórias da infância.",
    "Cientistas descobriram uma nova espécie de peixe em águas profundas.",
    "A final da Copa foi decidida nos pênaltis, com muita emoção na torcida.",
    "O conceito de buracos negros desafia nossa compreensão do espaço-tempo.",
    "O artista usou realidade aumentada para criar uma exposição interativa.",
    "A meditação diária ajuda a reduzir o estresse e aumentar a concentração."
]
```

2. Exemplo de funcionamento. Inclua os **scores** no output:

```
?> python3 embeddings.py "Ciência"
Cientistas descobriram uma nova espécie de peixe em águas profundas. (score: 0.2993)
O conceito de buracos negros desafia nossa compreensão do espaço-tempo. (score: 0.2129)
O artista usou realidade aumentada para criar uma exposição interativa. (score: 0.2085)
```



Utilize o modelo ‘paraphrase-multilingual-MiniLM-L12-v2’ por apresentar boa qualidade e suporte multi-linguístico, mas fique à vontade para explorar diferentes modelos.

Capítulo IV

	Exercício : 04
Retrieval-Augmented Generation	
Pasta de entrega : <i>ex04/</i>	
Arquivos para entregar : rag.py	
Funções ou bibliotecas autorizadas : Todas	

RAG (Retrieval-Augmented Generation) é uma abordagem que combina recuperação de informações e geração de texto. Em vez de depender apenas de um modelo de linguagem para gerar respostas, o RAG busca informações relevantes em um conjunto de documentos e as utiliza para melhorar a qualidade e a precisão das respostas geradas. Essa técnica é especialmente útil em cenários onde o conhecimento factual é crucial.

1. Crie um programa que implemente RAG para responder perguntas sobre a empresa fictícia **Orbit Motordrones**.
2. Seu programa deve:
 - Carregar o conteúdo do arquivo `orbit_motordrones.txt`, como base de conhecimento.
 - Criar um embedding para cada linha do arquivo.
 - Ser capaz de recuperar as linhas mais relevantes para a pergunta feita.
 - Imprimir as linhas recuperadas.
 - Utilizar um modelo de linguagem para gerar uma resposta baseada na pergunta e nas linhas recuperadas.
3. Seu prompt deve estar bem estruturado, enviando sua pergunta, um papel específico e as linhas recuperadas.
4. Exemplo de funcionamento:

```
python3 rag.py "Qual o prazo de entrega do orbit family?"
---- Linhas relevantes recuperadas: ----
- As primeiras entregas do Orbit Family estão previstas para logo após o início das pré-vendas em maio de 2026.
```

- O lançamento do Orbit Family está programado para o próximo semestre, prometendo uma revolução no mercado de mobilidade.
- As entregas do Orbit Urban estão previstas para ocorrer em 12 a 16 semanas após a confirmação do pedido.

As primeiras entregas do Orbit Family estão previstas para logo após o início das pré-vendas em maio de 2026.



Neste exercício, não é necessário criar uma interface de chat, ou memória. Foque em implementar a funcionalidade de RAG para responder perguntas baseadas no conteúdo do arquivo fornecido.



Defina um critério para selecionar as linhas mais relevantes, mantendo um equilíbrio entre a quantidade de texto enviado e a relevância da informação.



Você pode utilizar a biblioteca pickle para armazenar os embeddings em disco, evitando a necessidade de recalculá-los a cada execução do programa, como no exemplo abaixo:

```
import pickle
import os

if not os.path.exists('embeddings.pkl'):
    texts = [...] # lista de linhas do arquivo
    embeddings = [...] # lista de embeddings gerados
    with open('embeddings.pkl', 'wb') as f:
        pickle.dump({'texts': texts, 'embeddings': embeddings}, f)
else:
    with open('embeddings.pkl', 'rb') as f:
        data = pickle.load(f)
        texts = data['texts']
        embeddings = data['embeddings']
```

Capítulo V

Entrega e Avaliação entre Pares

- Entregue seu projeto em seu repositório *Git* disponível na página do projeto na intranet.
- Apenas o trabalho dentro do seu repositório será avaliado durante a defesa. Não hesite em verificar os nomes de seus arquivos e pastas para garantir que estejam corretos.
- No horário da avaliação, o avaliador se dirigirá à estação de trabalho do aluno a ser avaliado para realizar os testes. Um clone do repositório deverá ser realizado em uma nova pasta, e estes são os arquivos que serão avaliados.